

Project Title:

“Hematovision: Advanced Blood Cell Classification Using Transfer Learning”

Team Members:

- SHAIK IMRAN (TEAM LEAD)
- GAJJALA TEJA SRI (TEAM MEMBER)
- KODALI UMA MAHESWARI (TEAM MEMBER)
- TENALI IVANJILI BLESSI (TEAM MEMBER)

1. Introduction

1.1 Project Overview:

HematoVision is an automated diagnostic framework designed to classify various types of blood cells using **Transfer Learning**. By leveraging pre-trained deep learning models, the project aims to reduce the manual labour and human error associated with traditional microscopic blood smears.

1.2 Purpose:

The purpose of this project is to develop a comprehensive web-based application that streamlines the process of health appointment booking for patients, doctors, and administrative staff. This system is designed to digitize and automate the appointment scheduling process, reduce manual overhead, minimize booking conflicts, and enhance the overall efficiency of healthcare services.

1.3 Goals:

- Provide an intuitive interface for patients to search for doctors and book appointments.
- Allow doctors to manage their availability and view upcoming schedules.
- Enable admins to monitor system activity, manage users, and generate reports.
- Ensure secure authentication and role-based access for all users.
- Improve patient satisfaction by reducing wait times and miscommunication.

1.4 Features:

1. Multi-Class Morphological Recognition

The system doesn't just identify "cells"; it distinguishes between specific sub-types with high granularity.

- **WBC Differential:** Automatically categorizes White Blood Cells into **Neutrophils**, **Eosinophils**, **Basophils**, **Lymphocytes**, and **Monocytes**.
- **Abnormality Detection:** Potential to flag "blast cells" or "sickle cells," which are indicators of serious conditions like leukemia or anemia.

2. Optimized Transfer Learning Architectures

HematoVision leverages state-of-the-art neural networks to ensure high performance even with limited hardware.

- **Feature Extraction:** Uses pre-trained weights from models like **EfficientNet** or **MobileNetV2** for a balance between speed and accuracy.
- **Fine-Tuning Layers:** Custom-designed "top layers" (Dense and Dropout layers) that are specifically trained to interpret the colors and textures of Wright-Giemsa stained slides.

3. Automated Pre-processing Pipeline

To ensure the model isn't confused by lighting or slide quality, HematoVision includes:

- **Color Normalization:** Standardizes the purple/pink hues of the stain across different laboratory sources.
- **Image Segmentation:** Automatically crops individual cells from a wide-field microscopic image before classification.
- **Data Augmentation:** Increases model robustness by artificially creating variations (rotation, zoom, flipping) of the training images.

4. Explainability & Visualization (Grad-CAM)

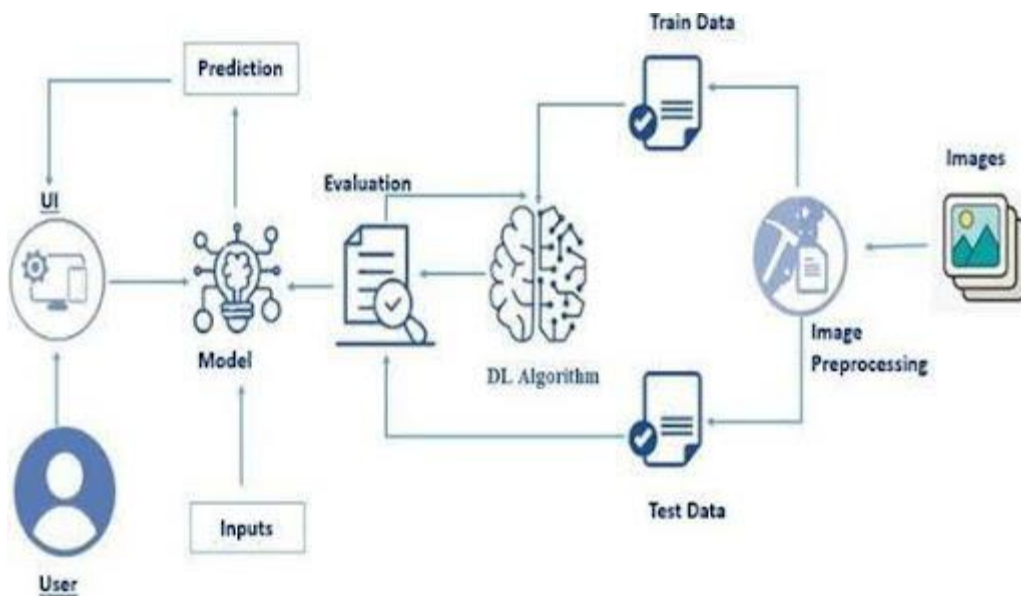
Trust is vital in medicine. This feature uses **Grad-CAM (Gradient-weighted Class Activation Mapping)** to highlight exactly *which* part of the cell the AI looked at to make its decision (e.g., the shape of the nucleus or the granules in the cytoplasm).

5. Performance Metrics Dashboard

The system provides a clear breakdown of its own reliability:

- **Confusion Matrices:** To show exactly where the model might be mixing up two similar cell types (like Monocytes and Lymphocytes).
- **Precision-Recall Curves:** Ensuring a low "false negative" rate for critical diagnoses.

2. ARCHITECTURE



3. PREREQUISITES

- To complete this project, you must require the following software, concepts, and packages
 - **Anaconda Navigator:**
 - Refer to the link below to download Anaconda Navigator
 - **Python packages:**
 - Open anaconda prompt as administrator
 - Type “pip install numpy” and click enter.
 - Type “pip install pandas” and click enter.
 - Type “pip install scikit-learn” and click enter.
 - Type ”pip install matplotlib” and click enter.
 - Type ”pip install scipy” and click enter.
 - Type ”pip install seaborn” and click enter.
 - Type ”pip install tensorflow” and click enter.
 - Type “pip install Flask” and click enter.

3.1 PRIOR KNOWLEDGE

You must have prior knowledge of the following topics to complete this project.

- DL Concepts
 - Neural Networks:: <https://www.analyticsvidhya.com/blog/2020/02/cnn-vs-rnn-vs-mlp-analyzing-3-types-of-neural-networks-in-deep-learning/>
- Deep Learning Frameworks:: <https://www.knowledgehut.com/blog/data-science/pytorch-vs-tensorflow>
- Transfer Learning: <https://towardsdatascience.com/a-demonstration-of-transfer-learning-of-vgg-convolutional-neural-network-pre-trained-model-with-c9f5b8b1ab0a>
- VGG16: <https://www.geeksforgeeks.org/vgg-16-cnn-model/>
- Convolutional Neural Networks (CNNs): <https://www.analyticsvidhya.com/blog/2021/05/convolutional-neural-networks-cnn/> s://www.javatpoint.com/k-nearest-neighbor-algorithm-for-machine-learning
- Overfitting and Regularization: <https://www.analyticsvidhya.com/blog/2021/07/prevent-overfitting-using-regularization-techniques/>
- Optimizers: <https://www.analyticsvidhya.com/blog/2021/10/a-comprehensive-guide-on-deep-learning-optimizers/>
- Flask Basics: https://www.youtube.com/watch?v=lj4I_CvBnt0

3.2 PROJECT OBJECTIVE

By the end of this project, you will:

- Know fundamental concepts and techniques used for Deep Learning.
- Gain a broad understanding of data.
- Have knowledge of pre-processing the data/transformation techniques on outliers and some visualization concepts.

3.3 PROJECT FLOW

- The user interacts with the UI (User Interface) to choose the image.
- The chosen image is analyzed by the model which is integrated with the flask application.
- Once the model analyses the input the prediction is showcased on the UI

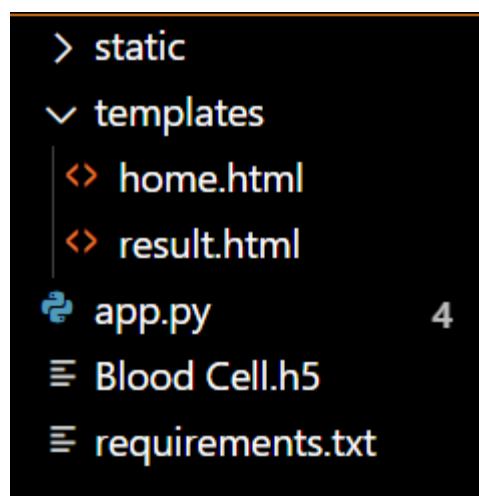
To accomplish this, we have to complete all the activities listed below,

- Data Collection: Collect or download the dataset that you want to train.
- Data pre-processing
 - Data Augmentation
 - Splitting data into train and test

- Model building
 - Import the model-building libraries
 - Initializing the model
 - Training and testing the model
 - Evaluating the performance of the model
 - Save the model
- Application Building
 - Create an HTML file
 - Build python code

4. PROJECT STRUCTURE

Create the Project folder which contains files as shown below



- We are building a Flask application with HTML pages stored in the templates folder and a Python script app.py for scripting.
- Blood Cell.h5 is our saved model. Further, we will use this model for flask integration.

5. DATA COLLECTION AND PREPARATION

ML depends heavily on data. It is the most crucial aspect that makes algorithm training possible. So, this section allows you to download the required dataset.

5.1 COLLECT THE DATASET

Download the dataset

There are many popular open sources for collecting the data. Eg: kaggle.com, UCI repository, etc.

This dataset contains 12,500 augmented images of blood cells (JPEG) with accompanying cell type labels (CSV). There are approximately 3,000 images for each of 4 different cell

types grouped into 4 different folders (according to cell type). The cell types are Eosinophil, Lymphocyte, Monocyte, and Neutrophil.

Link: <https://www.kaggle.com/datasets/paultimothymooney/blood-cells/data>

As the dataset is downloaded. Let us read and understand the data properly with the help of some visualization techniques and some analyzing techniques.

Note: There are several techniques for understanding the data. But here we have used some of it. In an additional way, you can use multiple techniques.

Activity 1.1: Importing the libraries:

Import the necessary libraries as shown in the image.

```
import os
import pandas as pd
import tensorflow as tf
from tensorflow import keras
import matplotlib.pyplot as plt
import seaborn as sns
import cv2
from tensorflow.keras.models import load_model
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input
from sklearn.metrics import confusion_matrix, accuracy_score
from sklearn.metrics import classification_report
from sklearn.model_selection import train_test_split
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

Activity 1.2: Read the Dataset:

- Our dataset format might be in .csv, excel files, .txt, .json, or zip files, etc. We can read the dataset with the help of pandas.

At first, unzip the data and convert it into a pandas data frame.

```

# Define the directory path
data_dir = 'C://Users//Dell//Downloads//BloodCells//dataset2-master//dataset2-master//images//TRAIN'

# Define the class labels
class_labels = ['eosinophil', 'lymphocyte', 'monocyte', 'neutrophil']

# Initialize lists to hold file paths and labels
filepaths = []
labels = []

# Loop through each class directory and gather file paths and labels
for label in class_labels:
    class_dir = os.path.join(data_dir, label)
    for file in os.listdir(class_dir):
        if file.endswith('.jpeg') or file.endswith('.png'): # Ensure file is an image
            filepaths.append(os.path.join(class_dir, file))
            labels.append(label)

# Create a DataFrame from the file paths and labels
bloodCell_df = pd.DataFrame({
    'filepaths': filepaths,
    'labels': labels
})

# Shuffle the DataFrame
bloodCell_df = bloodCell_df.sample(frac=1).reset_index(drop=True)

bloodCell_df.head()

```

	filepaths	labels
0	C://Users//Dell//Downloads//BloodCells//dataset2-master//dataset2-master//images//TRAIN//lymphocyte	lymphocyte
1	C://Users//Dell//Downloads//BloodCells//dataset2-master//dataset2-master//images//TRAIN//lymphocyte	lymphocyte

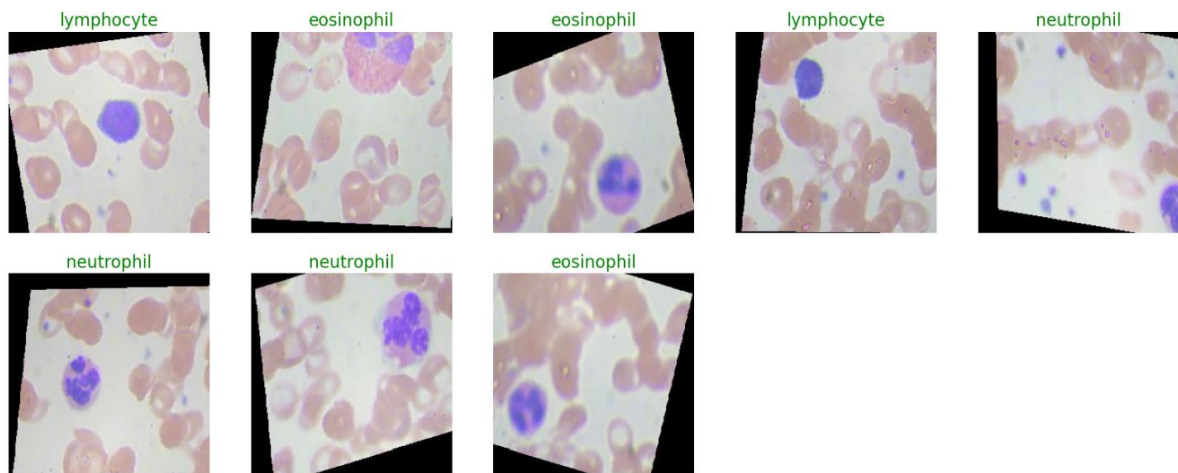
5.2 DATA VISUALIZATIONS

The provided Python code imports necessary libraries and modules for image manipulation. It selects a random image file from a specified folder path. Then, it displays the randomly selected image using IPython's Image module. This code is useful for showcasing random images from a directory for various purposes like data exploration or testing image processing algorithms.

```

import matplotlib.pyplot as plt
import numpy as np
def show_knee_images(image_gen):
    test_dict = test.class_indices
    classes = list(test_dict.keys())
    images, labels=next(image_gen)
    plt.figure(figsize=(20,20))
    length = len(labels)
    if length<25:
        r=length
    else:
        r=25
    for i in range(r):
        plt.subplot(5,5,i+1)
        image=(images[i]+1)/2
        plt.imshow(image)
        index=np.argmax(labels[i])
        class_name=classes[index]
        plt.title(class_name, color="green",fontsize=16)
        plt.axis('off')
    plt.show()
show_knee_images(train)

```



In the above code, I used class `ace of diamond` for prediction, This code randomly selects an image file from a specified folder (`folder_path`) containing JPEG, PNG, or JPEG files, and then displays the selected image using IPython's display function. It utilizes Python's OS and random modules for file manipulation and random selection, respectively. And It has predicted correctly as `ace of diamond`.

5.3 DATA AUGMENTATION

Data augmentation is a technique commonly employed in machine learning, particularly in computer vision tasks such as image classification, including projects like the BloodCells Classification . The primary objective of data augmentation is to artificially expand the size of the training dataset by applying various transformations to the existing images, thereby increasing the diversity and robustness of the data available for model training. This approach is particularly beneficial when working with limited labeled data.

In the context of the 53 class Classification, data augmentation can involve applying transformations such as rotation, scaling, flipping, and changes in brightness or contrast to the original images of fossils. These transformations help the model generalize better to variations and potential distortions present in real-world images, enhancing its ability to accurately classify unseen data.

This is a crucial step but this data is already cropped from the augmented data so. this time it is skipped accuracy is not much affected but the training time increased.

6. SPLIT DATA AND MODEL BUILDING

Train-Test-Split:

In this project, we have already separated data for training and testing.

```
train_images, test_images = train_test_split(bloodCell_df, test_size=0.3, random_state=42)
train_set, val_set = train_test_split(bloodCell_df, test_size=0.2, random_state=42)
```

```
print(train_set.shape)
print(test_images.shape)
print(val_set.shape)
print(train_images.shape)
```

```
(7965, 2)
(2988, 2)
(1992, 2)
(6969, 2)
```

```
image_gen = ImageDataGenerator(preprocessing_function= tf.keras.applications.mobilenet_v2.preprocess_input)
train = image_gen.flow_from_dataframe(dataframe= train_set,x_col="filepaths",y_col="labels",
                                     target_size=(244,244),
                                     color_mode='rgb',
                                     class_mode="categorical",
                                     batch_size=8,
                                     shuffle=False
                                     )
test = image_gen.flow_from_dataframe(dataframe= test_images,x_col="filepaths", y_col="labels",
                                    target_size=(244,244),
                                    color_mode='rgb',
                                    class_mode="categorical",
                                    batch_size=8,
                                    shuffle= False
                                    )
val = image_gen.flow_from_dataframe(dataframe= val_set,x_col="filepaths", y_col="labels",
                                   target_size=(244,244),
                                   color_mode= 'rgb',
                                   class_mode="categorical",
                                   batch_size=8,
                                   shuffle=False
                                   )
```

```
Found 7965 validated image filenames belonging to 4 classes.
Found 2988 validated image filenames belonging to 4 classes.
Found 1992 validated image filenames belonging to 4 classes.
```

6.1 MODEL BUILDING

Mobilenet V2 Transfer-Learning Model:

The MobileNetV2-based neural network is created using a pre-trained MobileNetV2 architecture with frozen weights. The model is built sequentially, incorporating the MobileNetV2 base, a flattening layer, dropout for regularization, and a dense layer with SoftMax activation for classification into four categories of blood cells. The model is compiled using the Adam optimizer and categorical cross-entropy loss. During training, which spans 5 epochs, a generator is employed for the training data, and validation is conducted with callbacks such as Model Checkpoint and Early Stopping. The best-performing model is saved as "blood_cell.h5" for future use. The model summary provides an overview of the architecture, showcasing the layers and parameters involved.

```

model = keras.models.Sequential([
    keras.layers.Conv2D(filters=128, kernel_size=(8, 8), strides=(3, 3), activation='relu', input_shape=(224, 224, 3)),
    keras.layers.BatchNormalization(),

    keras.layers.Conv2D(filters=256, kernel_size=(5, 5), strides=(1, 1), activation='relu', padding="same"),
    keras.layers.BatchNormalization(),
    keras.layers.MaxPool2D(pool_size=(3, 3)),

    keras.layers.Conv2D(filters=256, kernel_size=(3, 3), strides=(1, 1), activation='relu', padding="same"),
    keras.layers.BatchNormalization(),
    keras.layers.Conv2D(filters=256, kernel_size=(1, 1), strides=(1, 1), activation='relu', padding="same"),
    keras.layers.BatchNormalization(),
    keras.layers.Conv2D(filters=256, kernel_size=(1, 1), strides=(1, 1), activation='relu', padding="same"),
    keras.layers.BatchNormalization(),

    keras.layers.Conv2D(filters=512, kernel_size=(3, 3), activation='relu', padding="same"),
    keras.layers.BatchNormalization(),
    keras.layers.MaxPool2D(pool_size=(2, 2)),

    keras.layers.Conv2D(filters=512, kernel_size=(3, 3), activation='relu', padding="same"),
    keras.layers.BatchNormalization(),

    keras.layers.Conv2D(filters=512, kernel_size=(3, 3), activation='relu', padding="same"),
    keras.layers.BatchNormalization(),

    keras.layers.MaxPool2D(pool_size=(2, 2)),

    keras.layers.Conv2D(filters=512, kernel_size=(3, 3), activation='relu', padding="same"),
    keras.layers.BatchNormalization(),

    keras.layers.MaxPool2D(pool_size=(2, 2)),

    keras.layers.Flatten(),
    keras.layers.Dense(1024, activation='relu'),
    keras.layers.Dropout(0.5),
    keras.layers.Dense(1024, activation='relu'),
    keras.layers.Dropout(0.5),
    keras.layers.Dense(4, activation='softmax')
])

model.compile(
    loss='categorical_crossentropy',
    optimizer=tf.optimizers.SGD(learning_rate=0.001),
    metrics=['accuracy']
)

model.summary()

```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 73, 73, 128)	24704
batch_normalization (Batch Normalization)	(None, 73, 73, 128)	512
conv2d_1 (Conv2D)	(None, 73, 73, 256)	819456
batch_normalization_1 (Batch Normalization)	(None, 73, 73, 256)	1024
max_pooling2d (MaxPooling2D)	(None, 24, 24, 256)	0
conv2d_2 (Conv2D)	(None, 24, 24, 256)	590080
batch_normalization_2 (Batch Normalization)	(None, 24, 24, 256)	1024
conv2d_3 (Conv2D)	(None, 24, 24, 256)	65792
batch_normalization_3 (Batch Normalization)	(None, 24, 24, 256)	1024
conv2d_4 (Conv2D)	(None, 24, 24, 256)	65792
batch_normalization_4 (Batch Normalization)	(None, 24, 24, 256)	1024
conv2d_5 (Conv2D)	(None, 24, 24, 512)	1180160
batch_normalization_5 (Batch Normalization)	(None, 24, 24, 512)	2048
max_pooling2d_1 (MaxPooling2D)	(None, 12, 12, 512)	0
conv2d_6 (Conv2D)	(None, 12, 12, 512)	2359808
batch_normalization_6 (Batch Normalization)	(None, 12, 12, 512)	2048
conv2d_7 (Conv2D)	(None, 12, 12, 512)	2359808
batch_normalization_7 (Batch Normalization)	(None, 12, 12, 512)	2048

```
history = model.fit(train, epochs=5, validation_data=val, verbose=1)
```

Epoch 1/5

WARNING:tensorflow:From C:\Users\Dell\OneDrive\Documents\ANACONDA\Lib\site-packages\keras\src\utils\tf_utils.py:492: The name tf.ragged.RaggedTensorValue is deprecated. Please use tf.compat.v1.ragged.RaggedTensorValue instead.

WARNING:tensorflow:From C:\Users\Dell\OneDrive\Documents\ANACONDA\Lib\site-packages\keras\src\engine\base_layer_utils.py:384: The name tf.executing_eagerly_outside_functions is deprecated. Please use tf.compat.v1.executing_eagerly_outside_functions instead.

```
996/996 [=====] - 4064s 4s/step - loss: 1.6087 - accuracy: 0.3605 - val_loss: 1.0419 - val_accuracy: 0.5341
Epoch 2/5
996/996 [=====] - 3582s 4s/step - loss: 1.1049 - accuracy: 0.5171 - val_loss: 0.8389 - val_accuracy: 0.6401
Epoch 3/5
996/996 [=====] - 3307s 3s/step - loss: 0.8307 - accuracy: 0.6457 - val_loss: 0.5835 - val_accuracy: 0.7440
Epoch 4/5
996/996 [=====] - 6200s 6s/step - loss: 0.5703 - accuracy: 0.7602 - val_loss: 0.3648 - val_accuracy: 0.8529
Epoch 5/5
996/996 [=====] - 9178s 9s/step - loss: 0.3828 - accuracy: 0.8507 - val_loss: 0.2819 - val_accuracy: 0.8901
```

```
history1 = model.fit(train, epochs=1, validation_data=val, verbose=1)
```

```
996/996 [=====] - 3392s 3s/step - loss: 0.2661 - accuracy: 0.8925 - val_loss: 0.2942 - val_accuracy: 0.8800
```

7. TESTING MODEL & DATA PREDICTION

Evaluating the model

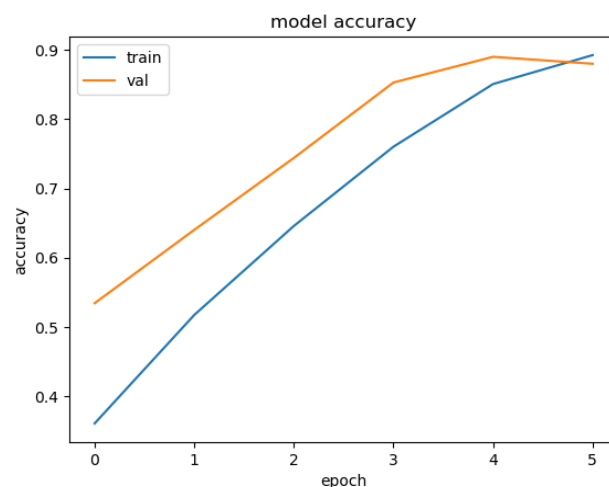
Here we have tested with the Mobilenet V2 Model With the help of the predict () function.

```
pred = model.predict(test)
pred = np.argmax(pred, axis=1) #pick class with highest probability

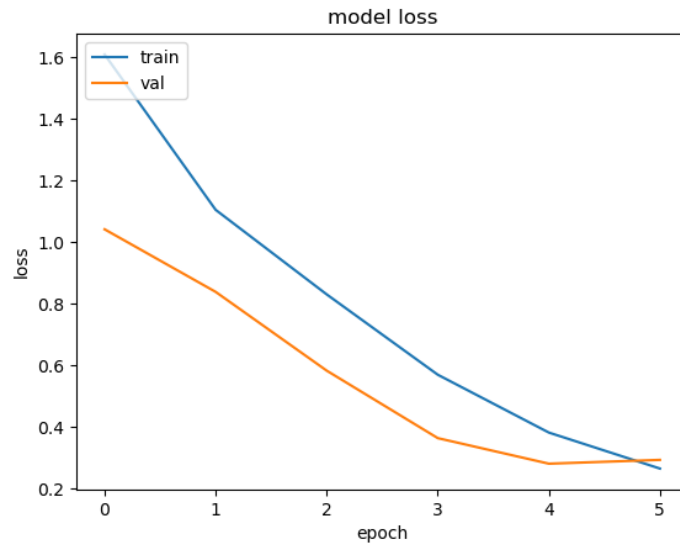
labels = (train.class_indices)
labels = dict((v,k) for k,v in labels.items())
pred2 = [labels[k] for k in pred]
```

```
374/374 [=====] - 332s 886ms/step
```

```
plt.plot(history.history['accuracy'] + history1.history['accuracy'])
plt.plot(history.history['val_accuracy'] + history1.history['val_accuracy'])
plt.title('model accuracy')
plt.ylabel('accuracy')
plt.xlabel('epoch')
plt.legend(['train', 'val'], loc='upper left')
plt.show()
```



```
plt.plot(history.history['loss'] + history1.history['loss'])
plt.plot(history.history['val_loss'] + history1.history['val_loss'])
plt.title('model loss')
plt.ylabel('loss')
plt.xlabel('epoch')
plt.legend(['train', 'val'], loc='upper left')
plt.show()
```



```
from sklearn.metrics import confusion_matrix, accuracy_score
from sklearn.metrics import classification_report

y_test = test_images.labels # set y_test to the expected output
print(classification_report(y_test, pred2))
print("Accuracy of the Model:", "{:.1f}%".format(accuracy_score(y_test, pred2)*100))
```

	precision	recall	f1-score	support
eosinophil	0.82	0.81	0.82	725
lymphocyte	0.90	0.99	0.94	762
monocyte	0.98	0.96	0.97	759
neutrophil	0.87	0.80	0.83	742
accuracy			0.89	2988
macro avg	0.89	0.89	0.89	2988
weighted avg	0.89	0.89	0.89	2988

Accuracy of the Model: 89.3%

```

import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix

class_labels = ['EOSINOPHIL', 'LYMPHOCYTE', 'MONOCYTE', 'NEUTROPHIL']

cm = confusion_matrix(y_test, pred2)

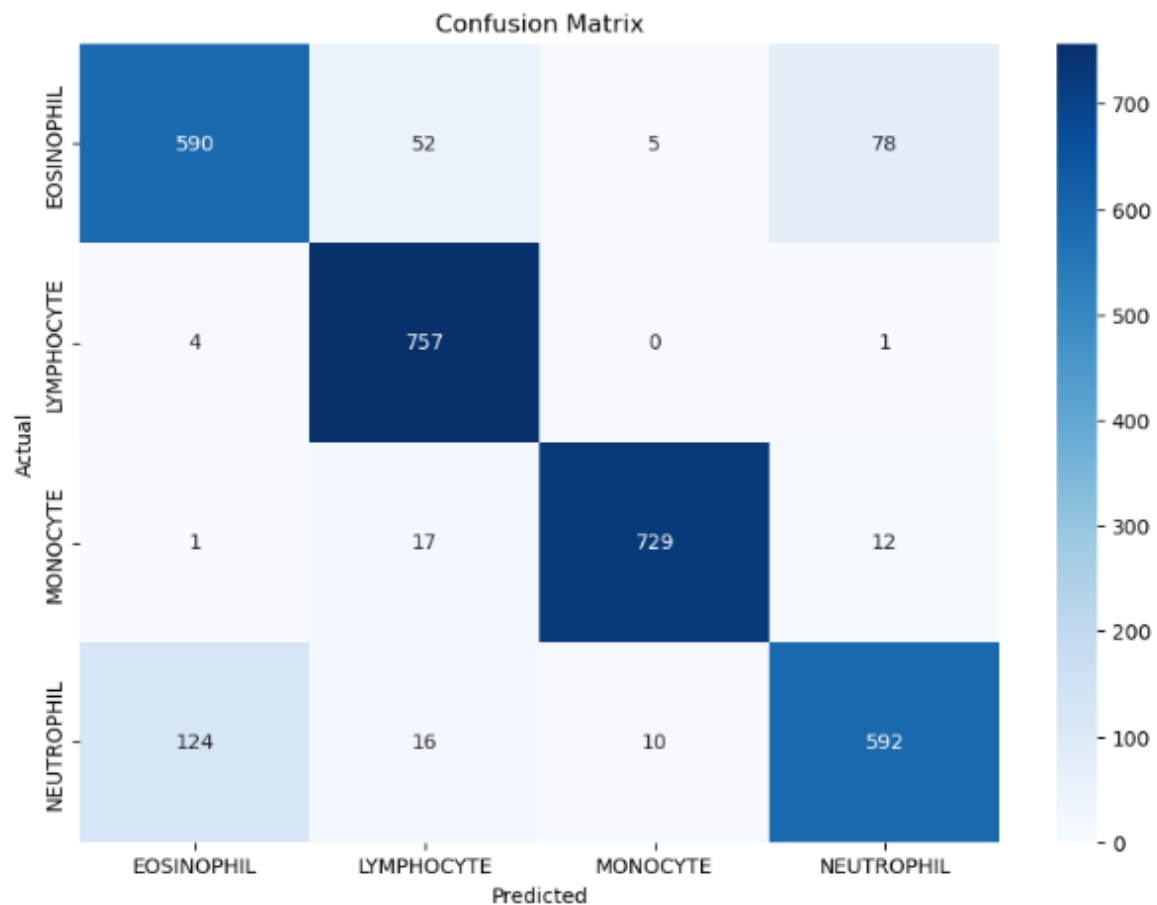
plt.figure(figsize=(10, 7))
sns.heatmap(cm, annot=True, fmt='g', vmin=0, cmap='Blues')

plt.xticks(ticks=[0.5, 1.5, 2.5, 3.5], labels=class_labels)
plt.yticks(ticks=[0.5, 1.5, 2.5, 3.5], labels=class_labels)
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.title("Confusion Matrix")

plt.show()

```



7.1 SAVING THE MODEL

Saving the model

Finally, we have chosen the best model now saving that model

```
model.save("Blood Cell.h5")
```

8. APPLICATION BUILDING

In this section, we will be building a web application that is integrated into the model we built. A UI is provided for the uses where he has to enter the values for predictions. The enter values are given to the saved model and prediction is showcased on the UI.

This section has the following tasks

- Building HTML Pages
- Building server-side script

8.1 BUILDING HTML PAGES

For this project create three HTML files namely

- home.html
- result.html

8.2 BUILD PYTHON CODE

Import the libraries

```
import os
import numpy as np
import cv2
from flask import Flask, request, render_template, redirect, url_for
from tensorflow.keras.models import load_model
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input
import matplotlib.pyplot as plt
import io
import base64

app = Flask(__name__)
model = load_model("Blood Cell.h5")
class_labels = ['eosinophil', 'lymphocyte', 'monocyte', 'neutrophil']
```

Load the saved model. Importing the Flask module in the project is mandatory. An object of the Flask class is our WSGI application. The Flask constructor takes the name of the current module (`__name__`) as argument.

Here we will be using the declared constructor to route to the HTML page which we have created earlier.

In the above example, the `/` URL is bound with the `index.html` function. Hence, when the index page of the web server is opened in the browser, the html page will be rendered.

Whenever you enter the values from the html page the values can be retrieved using POST Method.

Retrieves the value from UI:

```
app = Flask(__name__)
model = load_model("Blood_Cell.h5")
class_labels = ['eosinophil', 'lymphocyte', 'monocyte', 'neutrophil']

def predict_image_class(image_path, model):
    img = cv2.imread(image_path)
    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    img_resized = cv2.resize(img_rgb, (224, 224))
    img_preprocessed = preprocess_input(img_resized.reshape((1, 224, 224, 3)))
    predictions = model.predict(img_preprocessed)
    predicted_class_idx = np.argmax(predictions, axis=1)[0]
    predicted_class_label = class_labels[predicted_class_idx]
    return predicted_class_label, img_rgb
```

```
@app.route("/", methods=["GET", "POST"])
def upload_file():
    if request.method == "POST":
        if "file" not in request.files:
            return redirect(request.url)
        file = request.files["file"]
        if file.filename == "":
            return redirect(request.url)
        if file:
            file_path = os.path.join("static", file.filename)
            file.save(file_path)
            predicted_class_label, img_rgb = predict_image_class(file_path, model)

            # Convert image to string for displaying in HTML
            _, img_encoded = cv2.imencode('.png', cv2.cvtColor(img_rgb, cv2.COLOR_RGB2BGR))
            img_str = base64.b64encode(img_encoded).decode('utf-8')

            return render_template("result.html", class_label=predicted_class_label, img_data=img_str)
    return render_template("home.html")
```

Here we are routing our app to the `output()` function. This function retrieves all the values from the HTML page using a Post request. That is stored in an array. This array is passed to

the model. Predict () function. This function returns the prediction. This prediction value will be rendered to the text that we have mentioned in the output.html page earlier.

Main Function:

```
if __name__ == "__main__":  
    app.run(debug=True)
```

8.3 RUN THE WEB APPLICATION

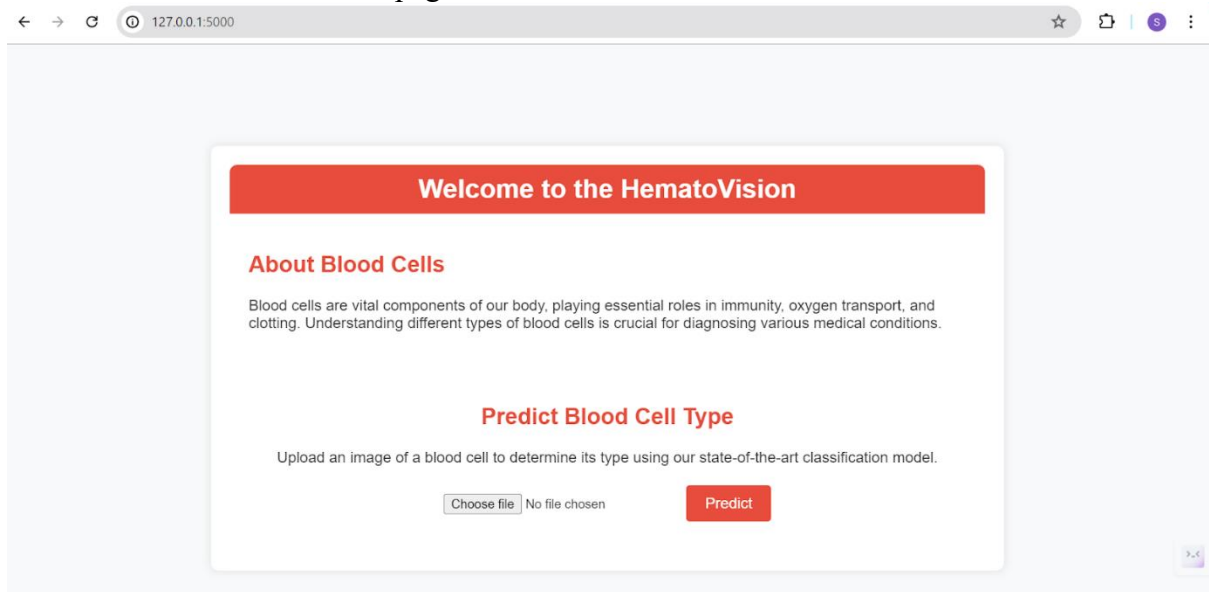
- : Run the application
 - Open Anaconda prompt from the start menu
 - Navigate to the folder where your Python script is.
 - Now type the “app.py” command
 - Navigate to the local host where you can view your web page.
 - Click on the inspect button from the top right corner, enter the inputs, click on the predict button, and see the result/prediction on the web.

```
* Serving Flask app 'app'  
* Debug mode: on  
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.  
* Running on http://127.0.0.1:5000  
Press CTRL+C to quit  
* Restarting with watchdog (windowsapi)
```

Now, Go to the web browser and write the localhost url (<http://127.0.0.1:5000>) to get the below results

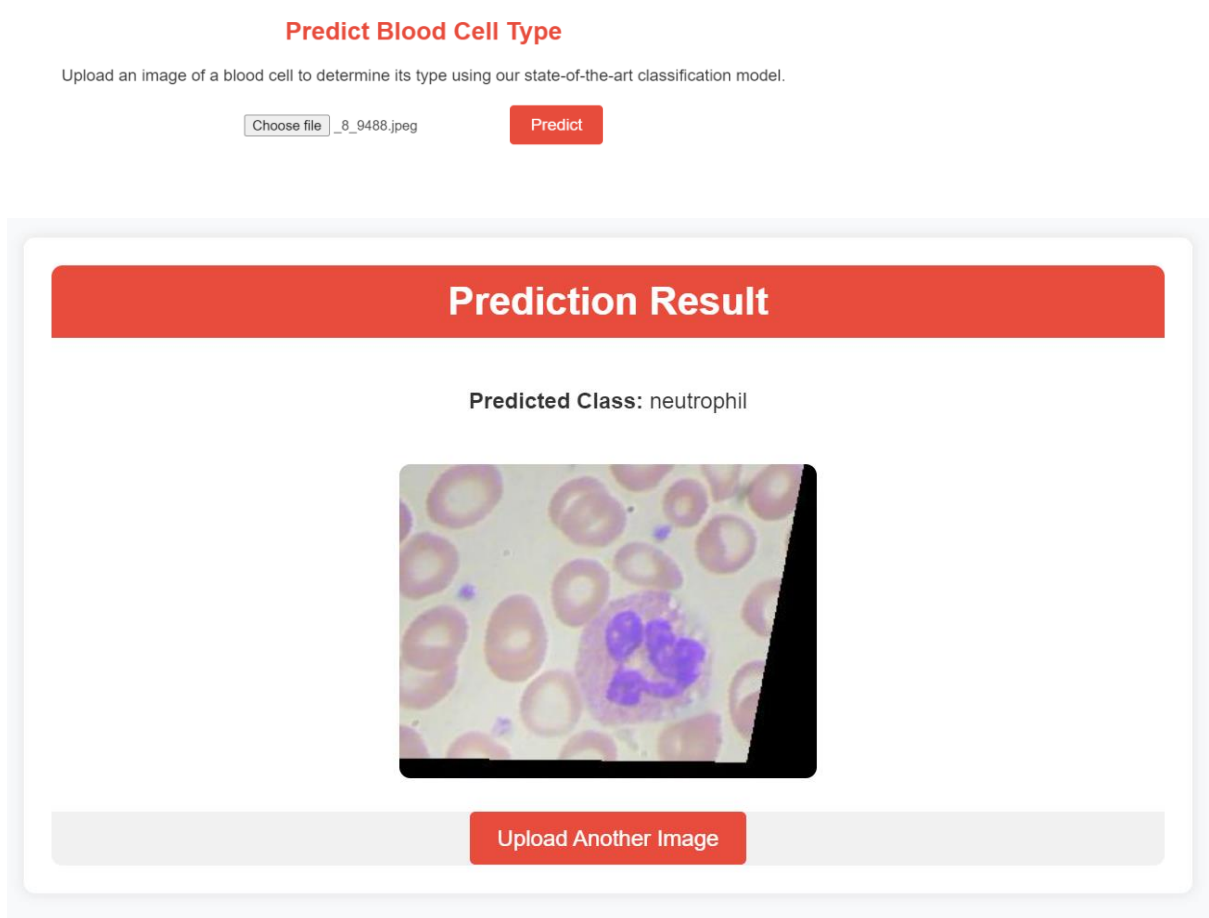
- UI Image preview:

Let's see what our index.html page looks like:



By clicking on choose file it will ask us to upload the image , then by clicking on the predict button , it will take us to the result.html

Test For Class-1 : Neutrophil



Test For Class-2 : Monocyt

Predict Blood Cell Type

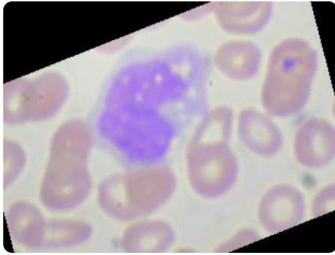
Upload an image of a blood cell to determine its type using our state-of-the-art classification model.

Choose file _3_9423.jpeg

Predict

Prediction Result

Predicted Class: monocyte



Upload Another Image

Test For Class-3 : Lymphocyte

Predict Blood Cell Type

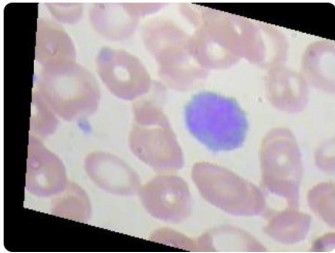
Upload an image of a blood cell to determine its type using our state-of-the-art classification model.

Choose file _5_9201.jpeg

Predict

Prediction Result

Predicted Class: lymphocyte



Upload Another Image

Test For Class-4 : Eosinophil

Predict Blood Cell Type

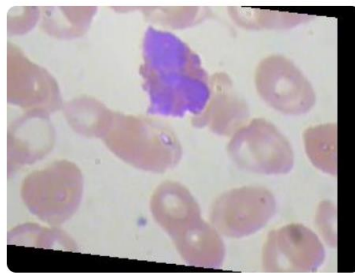
Upload an image of a blood cell to determine its type using our state-of-the-art classification model.

Choose file _3_9885.jpeg

Predict

Prediction Result

Predicted Class: eosinophil



Upload Another Image

9. CONCLUSION

HematoVision is an AI-based system that classifies different types of blood cells using deep learning and transfer learning. It helps reduce the time and effort required for manual microscopic analysis and improves accuracy in identifying white blood cells.

The project combines image preprocessing, a trained MobileNetV2 model, and a Flask web application to create an easy-to-use platform where users can upload blood cell images and get instant predictions. Performance evaluation methods like accuracy graphs and confusion matrices help measure how well the model works.

Overall, HematoVision shows how artificial intelligence can support medical professionals by making blood cell analysis faster, more efficient, and more reliable. In the future, the system can be expanded to detect more diseases and be used in real clinical environments.